DASH-07: Sharing and Reporting

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 7 of 7 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

A dashboard only delivers value when the right people can access it. This final notebook in the DASH series covers the full lifecycle of dashboard distribution — permission models, sharing with teams and stakeholders, scheduled reports via Dynatrace Workflows, exporting dashboard snapshots, managing dashboards as code through the Documents API, and version control patterns that keep your dashboard library maintainable.

Table of Contents

1. Permission Models
2. Sharing with Teams
3. Scheduled Reports via Workflows
4. Exporting Dashboard Snapshots
5. Dashboard as Code
6. Version Control Patterns
7. Summary and Series Wrap-Up

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`document:documents:write`, `document:direct-shares:write`, `automation:workflows:write`
API Access	For dashboard-as-code: a platform token or OAuth client with `document:documents:read` and `document:documents:write` (Monaco requires an OAuth client for documents — see §5). Dashboards are documents, not Settings objects
Prior Reading	DASH-01 through DASH-06

1. Permission Models

Dynatrace provides granular control over who can view, edit, and manage dashboards.

Dashboard Permission Levels

Level	Can View	Can Edit	Can Share	Can Delete
Viewer	Yes	No	No	No
Editor	Yes	Yes	No	No
Owner	Yes	Yes	Yes	Yes

Permission Assignment Strategies

Strategy	Description	Best For
Individual	Share directly with specific users	Small teams, sensitive dashboards
Group-based	Share with IAM groups	Department-wide dashboards
Environment-wide	Share with all environment users	Public status dashboards

Recommended Permission Model by Dashboard Tier

Tier	Owner	Editors	Viewers
Executive	Platform team lead	Platform team	All managers + leadership
Operations	SRE team	SRE + on-call engineers	Operations group
Engineering	Service team lead	Service team members	Engineering org

> **Tip:** Limit editor access to prevent well-intentioned modifications from breaking dashboards. Encourage teams to clone and customize rather than editing shared originals.

2. Sharing with Teams

Sharing Methods

The Dashboards app shares a dashboard three ways, all of them with Dynatrace users in your environment:

Method	How	Audience
Access for all	Share → *Visible to anyone in your environment (Read only)*	Every user in the environment, view only
Share access	Share → add users and user groups	Named users or groups, view or edit
Share link	Share → create a link for viewing or for editing	Anyone in the environment who has the link, at the level the link grants
Default for a team	There is no preset setting in the new Dashboards app — use *Access for all*, or a Launchpad as the team's landing page (FAQ-07)	New team members

Two classic-dashboard distribution features have no counterpart: **anonymous links** and **report subscriptions** do not exist in the new Dashboards app. Scheduled delivery is a

Workflow (§3). There is no embedding option for non-Dynatrace users.

> **Sources:**
>
> - Dashboards (DT docs) — *"Share links: Create links (URLs) pointing to your document and distribute the links through the channels of your choice (email, for example)."*
> - Share Dynatrace documents (DT docs) — *"No one outside your Dynatrace environment could use the link to access your document, of course, but anyone in your Dynatrace environment could use it."*
> - Upgrade from Dashboards Classic to Dashboards (DT docs) — *"don't exist in new Dashboards"* (said of both anonymous links and report subscriptions)

Dashboard Organization

As your dashboard library grows, organization becomes critical.

Practice	Description
Naming convention	`[Tier] - [Team/Service] - [Purpose]` e.g., "Ops - Checkout - Service Health"
Tags	Tag dashboards with tier, team, and service for easy filtering
Ownership registry	Maintain a list of dashboard owners for accountability
Regular cleanup	Archive dashboards that ran no tile queries in 90 days (query below)

Querying Dashboard Usage

There is no per-dashboard view log to query. The document audit trail — `event.kind == "AUDIT_EVENT"` with `event.provider == "DOCUMENTS"` in `dt.system.events` — records list reads and sharing changes, not reads of a dashboard: on a validation tenant (09/28/2026) its only event types over 30 days were `DIRECT_SHARES_LIST_READ`, `DIRECT_SHARE_RECIPIENTS_LIST_READ`, `DOCUMENTS_LIST_READ` and `ENV_SHARE_CREATE`. Use it to answer *who changed sharing*, not *who looked*.

The usable signal is query execution, so treat it as a **proxy**. Every query Dynatrace executes is stored as a `QUERY_EXECUTION_EVENT` in `dt.system.events`; for tile queries run by the Dashboards app, `client.application_context` is `dynatrace.dashboards` and `client.source` carries the dashboard URL. The query below counts those per

dashboard. A dashboard that never appears ran no tile queries in the window — the cleanup candidate. A count is queries, not views: a dashboard with many tiles produces many records per opening.

> **Corrected 09/28/2026.** An earlier version of this cell filtered `fetch events` on `event.kind == "AUDIT_LOG"`. No such kind exists, so it returned zero rows on every tenant — which the cleanup advice above would read as "no dashboard has been viewed".
>
> **Dictionary:** model `query_execution_event` (`data_object` `dt.system.events`), whose fields include `client.application_context` and `client.source` ; model `audit_event` (`dt.system.events`), read 09/28/2026.

```
// Dashboard usage proxy: tile queries run by the Dashboards app, per
dashboard, last 90 days.
// Document reads are not audited, so this counts query executions, not views.
// Dashboards with the oldest last_used — or that never appear — are cleanup
candidates.
fetch dt.system.events, from:-90d
| filter event.kind == "QUERY_EXECUTION_EVENT"
| filter client.application_context == "dynatrace.dashboards"
| parse client.source, "LD '/ui/dashboard/' LD:dashboard_id EOS"
| filter isNotNull(dashboard_id)
| summarize {queries = count(), viewers = countDistinct(user.id), last_used =
max(timestamp)}, by:{dashboard_id}
| sort last_used asc
```

3. Scheduled Reports via Workflows

Dynatrace Workflows can automate dashboard reporting — running DQL queries on a schedule and sending results via email, Slack, or other channels.

Workflow-Based Reporting Architecture

Component	Purpose
Trigger	Schedule (daily, weekly) or event-based
DQL Action	Run the same queries used in dashboard tiles

Component	Purpose
Format Action	Transform results into human-readable format
Notify Action	Send via email, Slack, Teams, PagerDuty

Example: Weekly Problem Summary Query

This query would be used in a Workflow DQL action for a weekly executive report.

```
// Weekly problem summary — suitable for automated report
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
// MTTR averages CLOSED problems only — avg() ignores the nulls the if()
returns for active ones
| summarize {total = count(), active = countIf(event.status == "ACTIVE"),
closed = countIf(event.status == "CLOSED"), avg_mttr_hours =
avg(if(event.status == "CLOSED", then: resolved_problem_duration / 1h))}
| fieldsAdd report_period = "Last 7 days"
```

Example: Daily Error Rate Summary

```
// Daily error rate by service — automated operations report
fetch spans, from:-24h
| filter span.kind == "server"
| summarize {total = count(), errors = countIf(span.status_code == "error")},
by:{dt.entity.service}
| fieldsAdd error_rate_pct = round(100.0 * errors / total, decimals: 2)
| filter error_rate_pct &gt; 1.0
| sort error_rate_pct desc
| limit 10
```

Example: Daily Log Volume Report

```
// Daily log volume by level — automated capacity report
fetch logs, from:-24h
| summarize log_count = count(), by:{loglevel}
| sort log_count desc
```

Workflow Scheduling Recommendations

Report Type	Schedule	Recipients
Executive summary	Weekly (Monday 8 AM)	Leadership, management
Operations daily	Daily (7 AM)	SRE team, on-call
SLA compliance	Monthly (1st of month)	Account management, leadership
Cost/volume tracking	Weekly	Platform team, finance

4. Exporting Dashboard Snapshots

Sometimes stakeholders need a static copy of a dashboard — for compliance, auditing, or offline review.

Export Options

Method	Format	Use Case
Screenshot	PNG/PDF	Quick sharing, email attachments
Dashboard JSON export	JSON	Backup, migration between environments
DQL results export	CSV	Data analysis in external tools
Workflow-generated report	Email/Slack message	Automated periodic snapshots

Dashboard JSON for Backup

Export the dashboard definition as JSON for version control or migration.

```
# Fetch a dashboard via the Documents API (platform token or OAuth client with document:documents:read)
curl -X GET "https://&lt;environment&gt;.apps.dynatrace.com/platform/document/v1/documents/&lt;dashboard-id&gt;" \
  -H "Authorization: Bearer &lt;platform-token&gt;" &gt; dashboard-backup.multipart
```

This endpoint returns **metadata and content together as a `multipart/form-data` response** — not a bare dashboard JSON file. The dashboard definition is the content part; the other part is the document's metadata. A Monaco `template:` or a restore needs the content part only. The Document service's content-only operation is `downloadDocumentContent` (*"Download latest document content."*); use it through the Document SDK or look up its REST path in your environment's API reference — this notebook has not verified that path.

> **Sources:** Document SDK (Dynatrace Developer) — `getDocument`: *"Return metadata and content in one multipart response."* The multipart response was observed on a validation tenant, 09/28/2026.
>
> **Note:** Dashboard export captures the definition (tiles, queries, variables) but not the current data. Re-importing will re-execute queries against the live environment.

5. Dashboard as Code

Managing dashboards as code enables version control, peer review, and automated deployment across environments.

Approaches

Approach	Tool	Best For
Documents API	REST API, curl, scripts	Simple backup and restore
Dynatrace Configuration as Code	Monaco CLI	Multi-environment deployment
Terraform Provider	Terraform	Infrastructure-as-code workflows

Validate the Payload Before You Merge It

> **A dashboard that fails validation does not load (SaaS 1.344, restated and tightened in SaaS 1.346).** SaaS 1.344 was published 07/27/2026 (rollout from 07/29/2026) and SaaS 1.346 restates the rule verbatim: *"Starting with Dynatrace version 1.346, Dynatrace applies stricter validation rules to dashboards and won't display dashboards that fail validation until you fix them."* Two sprints have now shipped this, so treat it as **current behavior** rather than something to wait for — while still confirming which version your tenant runs, because the enforcement arrives with the version. Before 1.344, a dashboard whose payload failed validation still loaded and surfaced validation warnings; now it does not load at all until fixed. Dynatrace names dashboards **created externally via API or by AI tooling** as the most affected population — which is precisely what every approach in the table above produces. Treat the warning state as a grace period, not a supported state — and note that 1.346 describes the rules as *stricter* again, so a payload that squeaked past 1.344 is not guaranteed to pass.
>
> **Gate every payload before you merge it:**
>
> 1. Deploy it to a **non-production tenant** and open the dashboard in the Dashboards app.
> 2. Confirm **zero validation warnings** — not "warnings we have decided to live with."
> 3. Prefer **exporting a working UI-authored dashboard** as your template over hand-writing the `tiles` / `layouts` map. The UI only emits shapes it can render.
>
> Note what this gate is *not*. `terraform plan`, a JSON linter, and Monaco's own config validation all check the surrounding configuration, not the dashboard document itself — to those tools the payload is an opaque blob. A `2xx` from the Documents API means the document was accepted for storage, not that it renders. The only check that answers the question is opening the dashboard.

The export → modify → review → deploy workflow described in §6 remains the working path. 1.344 raises the cost of skipping its validation step; it does not replace the workflow.

Monaco Configuration Example

```
# monaco/dashboards/config.yaml
configs:
  - id: ops-service-health
    type:
      document:
        kind: dashboard   # or "notebook" / "launchpad"
        private: false
    config:
      name: "Ops - Service Health"
      template: ops-service-health.json
      parameters:
        environment:
          type: environment
          name: DT_ENVIRONMENT
```

New dashboards are the Monaco `document` type (Monaco CLI 2.15.0+), not an `api:` type — `api:` selects classic configuration APIs. Parameters read environment variables with `type: environment` — a `{{ .Env.… }}` template string is not a documented Monaco 2 parameter form. Monaco authenticates to the Documents API with an **OAuth client**, not a classic API token.

Benefits of Dashboard as Code

Benefit	Description
Version control	Track changes, revert to previous versions
Peer review	Dashboard changes go through PR review
Multi-environment	Deploy same dashboard to dev, staging, prod
Disaster recovery	Rebuild entire dashboard library from code
Standardization	Enforce naming, layout, and variable conventions

> **Sources:**
>
> - What's new in Dynatrace SaaS 1.346 (DT docs) — the stricter dashboard-validation rule quoted above
> - What's new in Dynatrace SaaS 1.344 (DT docs) — the 1.344 release that first shipped it
> - Monaco configuration YAML file - list of type fields (DT docs) — *"Since Dynatrace Monaco CLI*

> *version 2.15.0+, the `document` type is supported, and it represents the API for Dashboards and Notebooks."*
>
> - Monaco configuration YAML file structure (DT docs) — *"The `environment` type parameter allows you to reference an environment variable."*
> - Monaco API support and access permission handling (DT docs) — *"OAuth credentials are required to target platform APIs"*

6. Version Control Patterns

Repository Structure for Dashboard Code

```
dashboards/
├── executive/
│   ├── business-kpi.json
│   └── sla-compliance.json
├── operations/
│   ├── service-health.json
│   ├── infrastructure.json
│   └── log-intelligence.json
├── engineering/
│   ├── service-deep-dive.json
│   ├── database-performance.json
│   └── deployment-impact.json
├── templates/
│   └── golden-signals.json
└── README.md
```

Change Management Process

Step	Action	Tool
1	Export current dashboard	Documents API
2	Modify in feature branch	Git
3	Review changes in PR	GitHub/GitLab
4	Deploy to staging	Monaco/Terraform

Step	Action	Tool
5	**Validate schema and render** — open the deployed dashboard in the Dashboards app on the staging tenant and confirm zero validation warnings	Dashboards app (staging tenant)
6	Review data fidelity — do the tiles show the numbers you expect, against the entities you expect?	Manual review
7	Deploy to production	Monaco/Terraform

Steps 5 and 6 are deliberately separate checks answering different questions. Step 5 asks *does this dashboard load and render at all* — a schema fault a human reviewer will not catch, because a tile can read perfectly in a pull request and still refuse to render. Step 6 asks *are the numbers right*, which only a person who knows the domain can judge. Collapsing them into one "validate in staging" step is how a payload with a malformed tile reaches production having passed review. See §5 for why the schema check now matters more than it used to.

Tracking Dashboard KPI Queries

Maintain a registry of which queries power which dashboards. This helps when DQL syntax changes or data sources are modified.

```
// Service count — useful for tracking monitoring coverage
fetch dt.entity.service
| summarize total_services = count()

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | summarize total_services = count()
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
```

```
// Host count — useful for tracking infrastructure coverage
fetch dt.entity.host
| summarize total_hosts = count()

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | summarize total_hosts = count()
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
```

7. Summary and Series Wrap-Up

In this notebook you learned:

- Dashboard permission levels and assignment strategies
- Methods for sharing dashboards with teams and stakeholders
- How to build automated reports using Dynatrace Workflows
- Dashboard export and snapshot options
- Dashboard-as-code approaches using Monaco and Terraform
- Version control patterns and change management processes

DASH Series Summary

Notebook	Key Takeaway
DASH-01	Dashboards vs notebooks, architecture, design principles
DASH-02	Three-tier hierarchy: Executive, Operations, Engineering
DASH-03	Executive KPIs: availability, MTTR, error budgets
DASH-04	Operations: real-time monitoring, log volume, problems
DASH-05	Engineering: traces, databases, endpoints, deployments
DASH-06	Variables, filters, template dashboard patterns
DASH-07	Sharing, reporting, dashboard-as-code, version control

With this series complete, you have the knowledge to build a comprehensive dashboard strategy — from executive summaries to engineering deep-dives, with variables for reusability and automation for reporting.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.